MCS-API Cluster Selection

Heads up of what’s happening upstream MCS-API wise

What this is about?

There is currently no standard way to prefer certains clusters in MCS-API

In Cilium we have a “Service Affinity” annotation
service.cilium.io/affinity: “local/remote/none”

But it’s fairly limited and there are some (old) CFP trying to improve it ->

KEP-5939 is addressing that in MCS-API (not yet merged, additional pair of eyes are welcomed!)

CFP: ClusterMesh global service cluster affinity #29200

Closed as not planned | Enhancement

dboslee opened on Nov 15, 2023 — Contributor

Cilium Feature Proposal

Is your proposed feature related to a problem?

Hey cilium folks thanks for the work you do on this project 🙌.

We use clustermesh/global services pretty heavily in our multi cluster environment. Our architecture is basically a cluster per region across several regions. Some of our services run in a cluster or two but not the others. We use global services in this case to reach those services when they aren't running in a given cluster.

Time and time again when deploying services to a subset of clusters I wish I had more control than simply `service.cilium.io/affinity: local|remote`. The only options are to route to local endpoints, remote endpoints, or all endpoints.

For low latency or for active/passive configurations this is not enough when you have more than 2 clusters.

As an example lets say we have endpoints in our clusters in us-east-1 and us-west-2 for the global service `backend`. When accessing this service from a cluster in `eu-central-1` there is no choice but to load balance requests across the endpoints in the other two regions. Ideally we could prefer to route traffic to the `us-east-1` cluster for lower latency.

We have worked around this in some cases in order to route traffic to a specific cluster by creating a `backend-us-east-1` global service that only has endpoints in the `us-east-1` cluster. The downside to this approach is that there is no failover to endpoints in other clusters if the endpoints in `us-east-1` goes down.

Describe the feature you'd like

I would like a way to specify cluster affinity. This would behave similar to the existing affinity annotation however instead you could specify which cluster to have affinity towards. When the endpoints in that cluster are not available the service will fall back to load balancing across any other available endpoints.

This could look something like `service.cilium.io/cluster-affinity: cluster-name`.

Create sub-issue | 7

The goal of MCS-API Cluster Selection

```yaml
apiVersion: multicluster.x-k8s.io/v1beta1
kind: ServiceExport
metadata:
  name: example
spec:
  clusterSelectorsFallbackPolicy: <AllClusters/None>
  clusterSelectors:
    - # ...  / First groups of clusters
    - # ...  / Second groups of clusters
    - # ... etc
```

This work similarly to “PreferSameZone” load balancing wise

The **chosen group** of clusters to load balance traffic to is the **first one with at least one endpoints ready or serving**

The fallback policy is to make cluster selectors open or close ended

Data source

```yaml
apiVersion: about.k8s.io/v1
kind: ClusterProperty
metadata:
  name: cluster.clusterset.k8s.io
spec:
  value: cluster-1
```

ClusterProperty is a CR from sig-multicluster to attach properties to Clusters.

A sort of “label” system for Clusters (with less restriction on values)

Selection criterias

Three type of selectors (only one type is allowed in each item):

- **LabelSelector** on ClusterProperty
- **CEL expression** on ClusterProperty (planned for a second iteration / currently looking for folks with CEL API design expertise to help!)
- **External decisions** via third party selectors on the selected group of clusters (planned for a third iteration or another separate KEP)
  - Could work alongside or within PlacementDecision CRDs currently used in tool helping to scheduling of Pods/Resources across clusters

A few examples with Label Selector

```yaml
apiVersion: multicluster.x-k8s.io/v1beta1
kind: ServiceExport
metadata:
  name: api
spec:
  clusterSelectors:
    - propertySelector:
        matchExpressions:
          - key: cluster.clusterset.k8s.io
            operator: In
            values:
              - "@SameAsImporter@"
    - propertySelector:
        matchLabels:
          region.topology.k8s.io: "@SameAsImporter@"
    - propertySelector:
        matchLabels:
          continent.topology.k8s.io: "@SameAsImporter@"
```

```yaml
apiVersion: multicluster.x-k8s.io/v1beta1
kind: ServiceExport
metadata:
  name: customer-data-api
spec:
  clusterSelectorsFallbackPolicy: None
  clusterSelectors:
    - propertySelector:
        matchLabels:
          compliance.mycompany.com: "gdpr"
```

Implementation in Cilium

Sharing ClusterProperties in Cluster Mesh

Share ClusterProperties in “cilium/state/clusterproperties/v1/<cluster>/<name>” and in the cilium-operator MCS-API controllers do this:

```yaml
apiVersion: multicluster.x-k8s.io/v1beta1
kind: ServiceImport
metadata:
  name: example
spec:
  clusterSelectorsFallbackPolicy: <AllClusters/None>
  clusterSelectors:
    - # ...  / First groups of clusters
    - # ...  / Second groups of clusters
    - # ... etc
```

→

```yaml
apiVersion: v1
kind: Service
metadata:
  name: derived-xxxxxx
  annotations:
    # ...
    service.cilium.io/cluster-affinity: "cluster1,cluster2|cluster1,cluster3|cluster5"
spec: # ...
```

Then the agents would pick the first group containing one endpoints ready or serving via the Cilium load balancer SelectBackends mechanism.

And Global Services gain a new useful feature in the process :D!

Bonus: coherence with Network Policy

As also mentioned in this other CFP and a bit at last year Cilium Dev Summit, ClusterProperty could also affect Cilium endpoints/identities.

Current implementation plan for that is to **watch local ClusterProperty CRs** when creating identity/endpoints **similarly to Namespace labels**.

The label on identity/endpoints could be for instance this:

io.cilium.k8s.clusterproperty.<property>: <value>

Those would then be naturally shared across clusters (note that we also need to have this working for local cluster endpoints in CNPs!)